2110573 PATTERN RECOGNITION

Homework 3: GMM

Computer Engineering, Chulalongkorn Univerity

Worralop Srichainont

February 11, 2026

1 Google Colab

For problem T1 - T4
https://colab.research.google.com/drive/1-KW-AA3R5MkhJRuyNt19wgAsJLrK15Iz

For problem T5 - T25
https://colab.research.google.com/drive/1btdl6-pEX-z6o5H9LKRP67yQYOlmTP1x

2110573 PATTERN RECOGNITION

Homework 03: GMM

- **Author:** Worralop Srichainont
- **Year:** 2025 (Semester 2)

Task: Hello Soft Clustering (GMM)

- **Problem**: T1 - T4

Dependencies

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import multivariate_normal
```

Problem `T1`

Using 3 mixtures, initialize your Gaussian with means (3,3), (2,2), and (-3,-3), and standard Covariance, I, the identity matrix. Use equal mix- ture weights as the initial weights. Repeat three iterations of EM. Write down $w_{n,j}$, m_j , $\vec{\mu_j}$, Σ_j for each EM iteration. (You may do the calculations by hand or write code to do so)

$$w_{n,j} = \frac{p(x_n; \vec{\mu_j}, \mathbf{\Sigma}_j)m_j}{\Sigma_j p(x_n; \vec{\mu_j}, \mathbf{\Sigma}_j)m_j} \tag{1}$$

$w_{n,j}$ means the probability that data point n comes from Gaussian number j.

Maximization: Update the model parameters, ϕ, $\vec{\mu_j}$, $\mathbf{\Sigma}_j$.

$$m_j = \frac{1}{N}\Sigma_n w_{n,j} \tag{2}$$

$$\vec{\mu_j} = \frac{\Sigma_n w_{n,j}\vec{x_n}}{\Sigma_n w_{n,j}} \tag{3}$$

$$\mathbf{\Sigma}_j = \frac{\Sigma_n w_{n,j}(\vec{x_n} - \vec{\mu_j})(\vec{x_n} - \vec{\mu_j})^T}{\Sigma_n w_{n,j}} \tag{4}$$

```
class GMM:
    def __init__(self, mixture_weight, mean_params, cov_params):
        """
        Initialize GMM.
        """
        # Copy construction values.
        self.mixture_weight = mixture_weight
        self.mean_params = mean_params
        self.cov_params = cov_params

        # Initiailize iteration.
        self.n_iter = 0

    def estimation_step(self, data):
        """
        Perform estimation step. Then, return w_{n,j} in from the equation 1
        """
        N = data.shape[0]
        K = len(self.mixture_weight)
        w = np.zeros((N, K))

```

```python
        for idx in range(K):
            pdf = multivariate_normal.pdf(
                data,
                mean=self.mean_params[idx],
                cov=self.cov_params[idx]
            )
            w[:, idx] = self.mixture_weight[idx] * pdf

        denom = np.sum(w, axis=1, keepdims=True)
        denom[denom == 0] = 1e-10
        w /= denom

        return w


    def maximization_step(self, data, w):
        """
        Perform maximization step. (Update parameters in this GMM model.)
        """
        N = data.shape[0]
        K = len(self.mixture_weight)
        N_cols = np.sum(w, axis=0)

        for idx in range(K):
            self.mixture_weight[idx] = N_cols[idx] / N

            weighted_sum = np.sum(w[:, idx].reshape(-1, 1) * data, axis=0)
            self.mean_params[idx] = weighted_sum / N_cols[idx]

            diff = data - self.mean_params[idx]
            numerator_cov = np.dot(w[:, idx] * diff.T, diff)
            new_cov = numerator_cov / N_cols[idx]
            self.cov_params[idx] = new_cov*np.eye(2)

    def get_log_likelihood(self, data):
        """
        Compute log likelihood.
        """
        K = len(self.mixture_weight)
        N = data.shape[0]
        weighted_pdfs = np.zeros((N, K))

        for idx in range(K):
            pdf = multivariate_normal.pdf(
                data,
                mean=self.mean_params[idx],
                cov=self.cov_params[idx]
            )
            weighted_pdfs[:, idx] = pdf * self.mixture_weight[idx]
            sum_weighted_pdfs = np.sum(weighted_pdfs, axis=1)
            log_prob = np.sum(np.log(sum_weighted_pdfs + 1e-10))

        return log_prob

    def print_iteration(self):
        print("m :\n", self.mixture_weight)
        print("mu :\n", self.mean_params)
        print("covariance matrix :\n", self.cov_params)
        print("------------------------------------------------------------")

    def perform_em_iterations(self, data, num_iterations, display=True):
        """
        Perform estimation & maximization steps with num_iterations.
        Then, return list of log_likelihood from those iterations.
        """
        log_prob_list = []

        # Display initialization.
        if display:
            print("Initialization")
            self.print_iteration()

        for n_iter in range(num_iterations):
```

```
            # Update parameters
            w = self.estimation_step(data)
            self.maximization_step(data, w)

            # Calculate log prob.
            log_prob = self.get_log_likelihood(data)
            log_prob_list.append(log_prob)

            # Display each iteration.
            if display:
                print(f"Iteration: {n_iter}")
                self.print_iteration()

        return log_prob_list
```

```
num_iterations = 3
num_mixture = 3
mixture_weight = [1] * num_mixture # m
mean_params = np.array([[3,3], [2,2], [-3,-3]], dtype = float)
cov_params = np.array([np.eye(2)] * num_mixture)

X, Y = np.array([1, 3, 2, 8, 6, 7, -3, -2, -7]), np.array([2, 3, 2, 8, 6, 7, -3, -4, -7])
data = np.vstack([X,Y]).T

gmm = GMM(mixture_weight, mean_params, cov_params)
log_prob_list = gmm.perform_em_iterations(data, num_iterations)
```

```
Initialization
m :
 [1, 1, 1]
mu :
 [[ 3.  3.]
 [ 2.  2.]
 [-3. -3.]]
covariance matrix :
 [[[1. 0.]
  [0. 1.]]

 [[1. 0.]
  [0. 1.]]

 [[1. 0.]
  [0. 1.]]]
------------------------------------------------------------
Iteration: 0
m :
 [np.float64(0.45757241940119386), np.float64(0.20909424706571345), np.float64(0.33333333353309275)]
mu :
 [[ 5.78992692  5.81887265]
 [ 1.67718211  2.14523106]
 [-4.         -4.66666666]]
covariance matrix :
 [[[4.53619412 0.        ]
  [0.         4.28700611]]

 [[0.51645579 0.        ]
  [0.         0.13152618]]

 [[4.66666668 0.        ]
  [0.         2.88888891]]]
------------------------------------------------------------
Iteration: 1
m :
 [np.float64(0.4071161831594476), np.float64(0.2595496084607245), np.float64(0.3333342083798279)]
mu :
 [[ 6.27176215  6.27262711]
 [ 1.72091544  2.14764812]
 [-3.99998589 -4.6666488 ]]
covariance matrix :
 [[[2.94672736 0.        ]
  [0.         2.93847196]]

 [[0.49649261 0.        ]
  [0.         0.12584815]]

 [[4.66673088 0.        ]
  [0.         2.88900236]]]
------------------------------------------------------------
Iteration: 2
m :
 [np.float64(0.36070908638871857), np.float64(0.3059567726926511), np.float64(0.33333414091863034)]
mu :
 [[ 6.6962644   6.69629468]
 [ 1.91071238  2.27383436]
 [-3.99998673 -4.6666501 ]]
```

Result

Initialization

- Mixing Coefficients: `[1.0000, 1.0000, 1.0000]`
- Mean :

```
[[ 3.0000  3.0000]
 [ 2.0000  2.0000]
 [-3.0000 -3.0000]]
```

- Covariance Matrix:

```
[[[1.0000 0.0000]
  [0.0000 1.0000]]

 [[1.0000 0.0000]
  [0.0000 1.0000]]

 [[1.0000 0.0000]
  [0.0000 1.0000]]]
```

Iteration 0

- Mixing Coefficients: `[0.4576, 0.2091, 0.3333]`
- Mean :

```
[[ 5.7899  5.8188]
 [ 1.6772  2.1452]
 [-4.0000 -4.6667]]
```

- Covariance Matrix:

```
[[[4.5362 0.0000]
  [0.0000 4.2870]]

 [[0.5165 0.0000]
  [0.0000 0.1315]]

 [[4.6667 0.0000]
  [0.0000 2.8889]]]
```

Iteration 1

- Mixing Coefficients: `[0.4071, 0.2595, 0.3333]`
- Mean :

```
[[ 6.2718  6.2726]
 [ 1.7209  2.1476]
 [-4.0000 -4.6667]]
```

- Covariance Matrix:

```
[[[2.9467 0.0000]
  [0.0000 2.9385]]

 [[0.4965 0.0000]
  [0.0000 0.1258]]
```

```
 [[4.6667 0.0000]
  [0.0000 2.8889]]]
```

Iteration 2

- Mixing Coefficients: `[0.3607, 0.3060, 0.3333]`
- Mean :

```
[[ 6.6963  6.6963]
 [ 1.9107  2.2738]
 [-4.0000 -4.6667]]
```

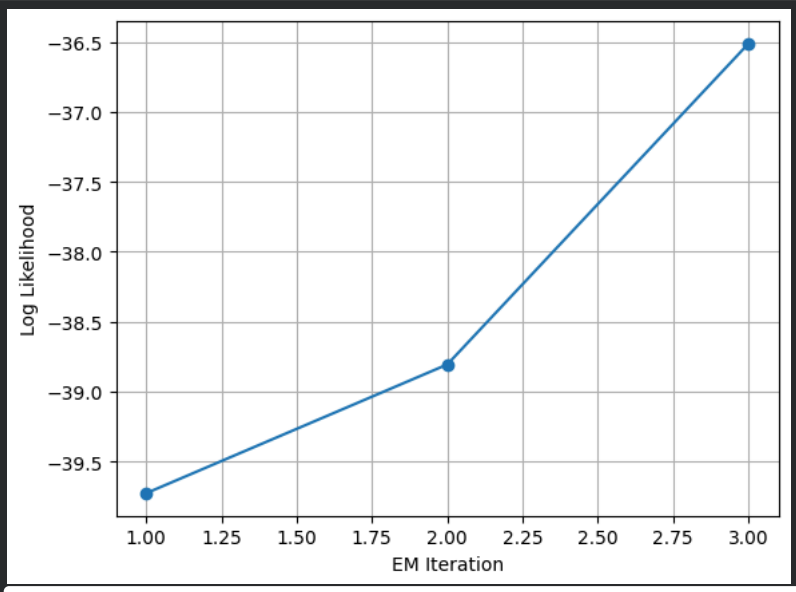
- Covariance Matrix:

```
[[[1.7396 0.0000]
  [0.0000 1.7393]]

 [[0.6290 0.0000]
  [0.0000 0.1988]]

 [[4.6667 0.0000]
  [0.0000 2.8889]]]
```

Problem T2

Plot the log likelihood of the model given the data after each EM step. In other words, plot $\log \prod_n p(\vec{x_n}|\varphi, \vec{\mu}, \Sigma)$. Does it goes up every iteration just as we learned in class?

```
X_RANGE = range(1, num_iterations + 1)

plt.figure()
plt.plot(X_RANGE, log_prob_list, marker="o")

plt.xlabel("EM Iteration")
plt.ylabel("Log Likelihood")
plt.grid(True)

plt.show()
```



ANSWER: As we can see, the value increases for every iterations.

Problem T3

Using 2 mixtures, initialize your Gaussian with means (3,3) and (-3,-3), and standard Covariance, I, the identity matrix. Use equal mixture weights as the initial weights. Repeat three iterations of EM. Write down $w_{n,j}$, m_j , $\vec{\mu_j}$, Σ_j for each EM iteration.

```
num_mixture = 2
mixture_weight = [1] * num_mixture

mean_params = np.array([[3,3], [-3,-3]], dtype = float)
cov_params = np.array([np.eye(2)] * num_mixture)

gmm2 = GMM(mixture_weight, mean_params, cov_params)
log_prob_list2 = gmm2.perform_em_iterations(
    data, num_iterations=3
)
```

```
Initialization
m :
 [1, 1]
mu :
 [[ 3.  3.]
 [-3. -3.]]
covariance matrix :
 [[[1. 0.]
  [0. 1.]]

 [[1. 0.]
  [0. 1.]]]
------------------------------------------------------------
Iteration: 0
m :
 [np.float64(0.6666666649702521), np.float64(0.3333333350297478)]
mu :
 [[ 4.50000001  4.66666667]
 [-3.99999997 -4.66666663]]
covariance matrix :
 [[[6.91666665 0.        ]
  [0.         5.88888889]]

 [[4.66666677 0.        ]
  [0.         2.8888891 ]]]
------------------------------------------------------------
Iteration: 1
m :
 [np.float64(0.6666943621060054), np.float64(0.33330563789399464)]
mu :
 [[ 4.49961311  4.66620178]
 [-3.99993241 -4.66651231]]
covariance matrix :
 [[[6.91944755 0.        ]
  [0.         5.89275124]]

 [[4.66806942 0.        ]
  [0.         2.89103318]]]
------------------------------------------------------------
Iteration: 2
m :
 [np.float64(0.6666945259520648), np.float64(0.3333054740479351)]
mu :
 [[ 4.49961084  4.66619903]
 [-3.99993206 -4.66651141]]
covariance matrix :
 [[[6.91946372 0.        ]
  [0.         5.8927741 ]]

 [[4.66807754 0.        ]
  [0.         2.89104566]]]
------------------------------------------------------------
```

Result

Initialization

- Mixing Coefficients: `[1.0000, 1.0000]`
- Mean :

```
[[ 3.0000   3.0000]
 [-3.0000  -3.0000]]
```

- Covariance Matrix:

```
[[[1.0000 0.0000]
  [0.0000 1.0000]]

 [[0.5165 0.0000]
  [0.0000 0.1315]]]
```

Iteration 0

- Mixing Coefficients: `[0.6667, 0.3333]`
- Mean :

```
[[ 4.5000   4.6667]
 [-4.0000  -4.6667]]
```

- Covariance Matrix:

```
[[[6.9167 0.0000]
  [0.0000 5.8889]]

 [[4.6667 0.0000]
  [0.0000 2.8889]]]
```

Iteration 1

- Mixing Coefficients: `[0.6667, 0.3333]`
- Mean :

```
[[ 4.4996   4.6662]
 [-4.0000  -4.6665]]
```

- Covariance Matrix:

```
[[[6.9195 0.0000]
  [0.0000 5.8928]]

 [[4.6681 0.0000]
  [0.0000 2.8910]]]
```

Iteration 2

- Mixing Coefficients: `[0.6667, 0.3333]`
- Mean :

```
[[ 4.4996   4.6662]
 [-4.0000  -4.6665]]
```

- Covariance Matrix:

```
[[[6.9195 0.0000]
  [0.0000 5.8928]]

 [[4.6681 0.0000]
  [0.0000 2.8910]]]
```

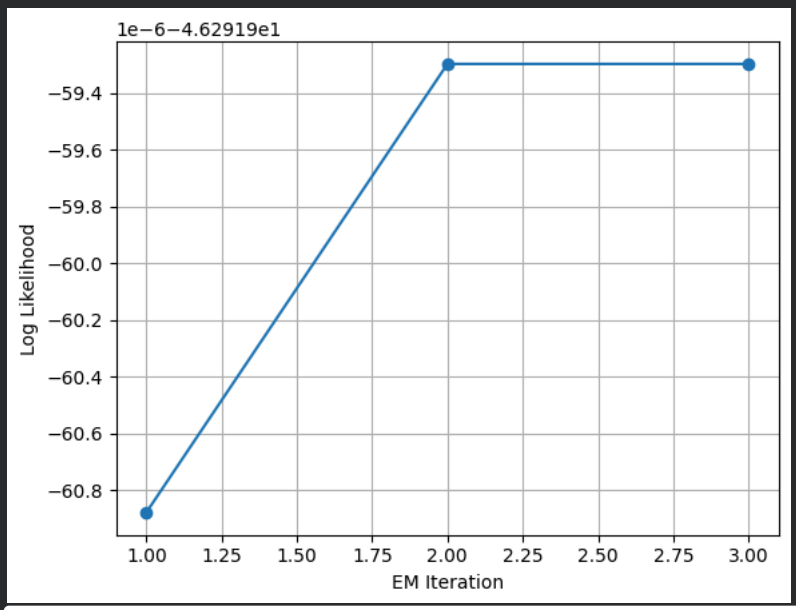
Problem `T4`

Plot the log likelihood of the model given the data after each EM step. Compare the log likelihood between using two mixtures and three mixtures. Which one has the better likelihood?

Plot `log_likelihood` From `T3`

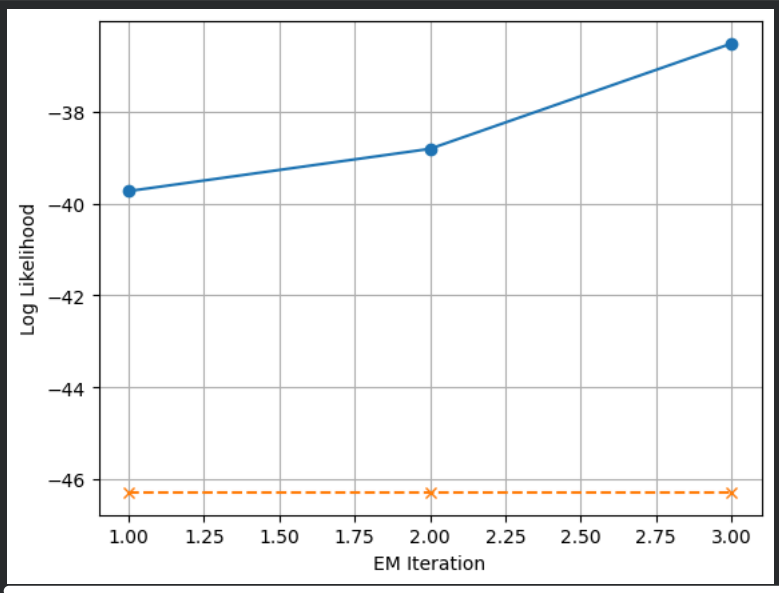
```
X_RANGE = range(1, len(log_prob_list2) + 1)

plt.figure()
plt.plot(X_RANGE, log_prob_list2, marker="o")
```

```
plt.xlabel("EM Iteration")
plt.ylabel("Log Likelihood")

plt.grid(True)
plt.show()
```



Plot `log_likelihood` From `T1` and `T3`

```
X_RANGE_T1 = range(1, len(log_prob_list) + 1)
X_RANGE_T3 = range(1, len(log_prob_list2) + 1)

plt.figure()
plt.plot(X_RANGE_T1, log_prob_list, marker="o")
plt.plot(X_RANGE_T3, log_prob_list2, marker="x", linestyle="--")

plt.xlabel("EM Iteration")
plt.ylabel("Log Likelihood")

plt.grid(True)
plt.show()
```



Display log likelihood of `T1` and `T3`

```
t1_log_likelihood = log_prob_list[-1]
t3_log_likelihood = log_prob_list2[-1]

print(f"T1: 3 Mixtures Log Likelihood: {t1_log_likelihood:.4f}")
print(f"T3: 2 Mixtures Log Likelihood: {t3_log_likelihood:.4f}")
```

```
T1: 3 Mixtures Log Likelihood: -36.5130
T3: 2 Mixtures Log Likelihood: -46.2920
```

ANSWER: The model with 3 mixtures has better likelihood

2110573 PATTERN RECOGNITION

Homework 03: GMM

- **Author:** Worralop Srichainont
- **Year:** 2025 (Semester 2)

Task: Fisherface

- **Problem**: T5 - T25

Dependencies

```
import scipy.io
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from skimage import img_as_float
```

Data Resources

```
!wget -nc "https://drive.google.com/uc?id=18Vaza8W-IytHoZosPiYZP9dhyZcyWt7-" -O facedata_mat.zip
!unzip -o facedata_mat.zip
```

```
File ‘facedata_mat.zip’ already there; not retrieving.
Archive:  facedata_mat.zip
  inflating: facedata.mat
```

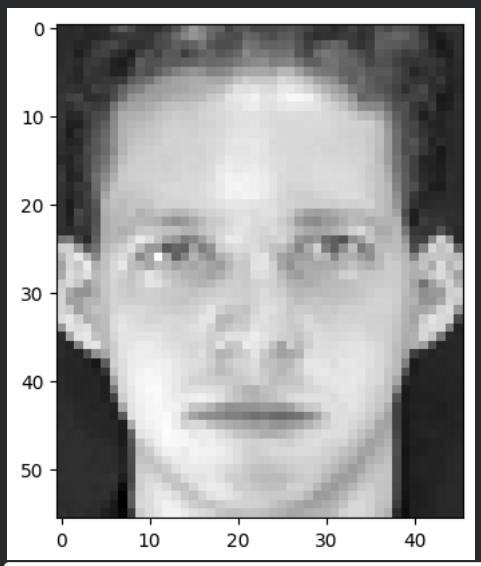
```
FILE_PATH = "facedata.mat"
```

```
data = scipy.io.loadmat(FILE_PATH)
data_size = data['facedata'].shape
```

Preprocessing xf

```
xf = np.zeros((data_size[0], data_size[1], data['facedata'][0,0].shape[0], data['facedata'][0,0].shape[
for i in range(data['facedata'].shape[0]):
    for j in range(data['facedata'].shape[1]):
        xf[i,j] = img_as_float(data['facedata'][i,j])
```

```
plt.imshow(xf[0,0], cmap = 'gray')
plt.show()
```



Problem T5

What is the Euclidean distance between xf[0,0] and xf[0,1]? What is the Euclidean distance between xf[0,0] and xf[1,0]? Does the numbers make sense? Do you think these numbers will be useful for face verification?

```
def L2_dist(x1, x2):
    """
    Calculate L2 distance.
    """
    return np.sqrt(np.sum((x1 - x2) ** 2))
```

```
def test_L2_dist():
    assert L2_dist(np.array([1, 2, 3]), np.array([1, 2, 3])) == 0.0
    assert L2_dist(np.array([0, 0, 0]), np.array([1, 2, 3])) == np.sqrt(14)

test_L2_dist()
```

```
dist_same_person = L2_dist(xf[0,0], xf[0,1])
dist_diff_person = L2_dist(xf[0,0], xf[1,0])

print(f'Euclidean distance between xf[0, 0] and xf[0, 1] is {dist_same_person}')
print(f'Euclidean distance between xf[0, 0] and xf[1, 0] is {dist_diff_person}')
```

```
Euclidean distance between xf[0, 0] and xf[0, 1] is 10.037616294165492
Euclidean distance between xf[0, 0] and xf[1, 0] is 8.173295099737281
```

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))

# Subplot for the first image
axes[0].imshow(xf[0, 0], cmap="gray")
axes[0].axis("off")
axes[0].set_title("Reference Image")

# Subplot for the second image
axes[1].imshow(xf[0, 1], cmap="gray")
axes[1].axis("off")
axes[1].set_title(f"Same Person\nDistance = {dist_same_person:.4f}")

# Subplot for the third image
axes[2].imshow(xf[1, 0], cmap="gray")
axes[2].axis("off")
axes[2].set_title(f"Different Person\nDistance = {dist_diff_person:.4f}")

# Plot images
plt.show()
```

Answer

In terms of mathematical, it makes sense because different image should have distinct distance.

But in reality, sometimes it is nonsense, because different lighting and camera angle have more effect than the face structure itself.

Problem `T6`

Write a function that takes in a set of feature vectors T and a set of feature vectors D, and then output the similarity matrix A. Show the matrix as an image.

Use the feature vectors from the first 3 images from all 40 people for list T (in order x[0, 0], x[0, 1], x[0, 2], x[1, 0], x[1, 1], ...x[39, 2]).

Use the feature vectors from the remaining 7 images from all 40 people for list D (in order x[0, 3], x[0, 4], x[0, 5], x[1, 6], x[0, 7], x[0, 8], x[0, 9], x[1, 3], x[1, 4]...x[39, 9]).

We will treat T as our training images and D as our testing images

```
def organize_shape(matrix):
    """
    Reduce matrix dimension of 2D image to 1D and merge people and image dimension.
    This function can be useful at organizing matrix shapes.

    Example:
        Input shape: (people_index, image_index, image_shape[0], image_shape[1])
        Output shape: (people_index*image_index, image_shape[0]*image_shape[1])
    """
    n_people, n_images, height, width = matrix.shape

    n_rows = n_people * n_images
    n_cols = height * width

    return matrix.reshape(n_rows, n_cols)
```

```
def generate_similarity_matrix(A, B):
    """
    Calculate similarity matrix M,
    which M[i, j] is a distance between A[i] and B[j].
    """
    diff = A[:, np.newaxis, :] - B[np.newaxis, :, :]
    similarity_matrix = np.sqrt(np.sum(diff ** 2, axis=-1))

    return similarity_matrix
```

```
def test_generate_similarity_matrix():
    test_A = np.array([[1, 2],[3,4]])
    test_B = np.array([[1, 2], [5, 6], [7, 8]])
    expected_matrix = np.sqrt(np.array([[0, 32, 72], [8, 8, 32]]))
    assert (generate_similarity_matrix(test_A, test_B) == expected_matrix).all()

test_generate_similarity_matrix()
```

Show Similarity Matrix

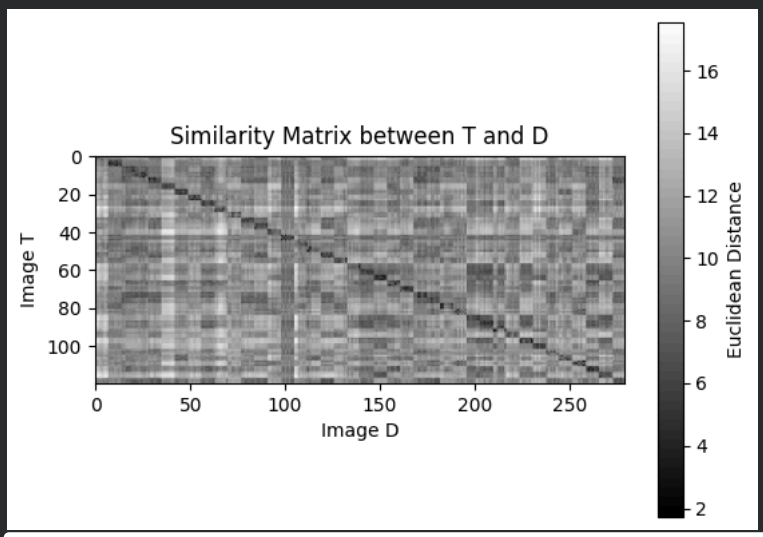
Calculate similarity matrix between `T` and `D`.

```
T_raw = xf[:, 0:3]
D_raw = xf[:, 3:10]

T_vector = organize_shape(T_raw)
D_vector = organize_shape(D_raw)

similarity_matrix = generate_similarity_matrix(
    T_vector, D_vector
)
```

Plot similarity matrix between `T` and `D`.

```
plt.figure()

plt.imshow(similarity_matrix, cmap="gray")
plt.colorbar(label="Euclidean Distance")

plt.title("Similarity Matrix between T and D")
plt.xlabel("Image D")
plt.ylabel("Image T")
```

```
plt.show()
```



Problem T7

From the example similarity matrix above, what does the black square between [5:10,5:10] suggest about the pictures from person number 2?

What do the patterns from person number 1 say about the images from person 1?

The black square from person number 2 in the similarity matrix means both images are similar.

Person number 1 has gray patterns, because it is not so similarity.

Problem T8

Write a function that takes in the similarity matrix created from the previous part, and a threshold t as inputs.

The outputs of the function are the true positive rate and the false alarm rate of the face verification task (280 Test images, tested on 40 people, a total of 11200 testing per threshold).

What is the true positive rate and the false alarm rate for t = 10?

```
def evaluate_performance(similarity_matrix, threshold):
    """
    Calculate true positive rate and false alarm rate from given similarity_matrix and threshold.
    """
    n_test_images = similarity_matrix.shape[1]
    n_people = 40

    new_similarity_matrix = similarity_matrix.reshape(
        n_people, 3, n_test_images
    )

    min_dists = np.min(new_similarity_matrix, axis=1)

    pred = min_dists < threshold
    truth = np.zeros(
        (n_people, n_test_images), dtype=bool
    )

    for idx in range(n_people):
        begin_col = 7 * idx
        end_col = 7 * (idx + 1)
        truth[idx, begin_col : end_col] = True

    positives = np.sum(truth)
    negatives = np.sum(~truth)
    true_positives = np.sum(pred & truth)
    false_positives = np.sum(pred & ~truth)

    true_pos_rate = true_positives / positives
    false_pos_rate = false_positives / negatives

```

```
    return true_pos_rate, false_pos_rate


evaluate_performance(similarity_matrix, 9.5)
```

```
(np.float64(0.9928571428571429), np.float64(0.33507326007326005))
```

```
THRESHOLD = 10.0

true_pos_rate, false_pos_rate = evaluate_performance(
    similarity_matrix, THRESHOLD
)

print(f"Threshold = {THRESHOLD}")
print(f"- True Positive Rate: {true_pos_rate:.4f}")
print(f"- False Positive Rate: {false_pos_rate:.4f}")
```

```
Threshold = 10.0
- True Positive Rate: 0.9964
- False Positive Rate: 0.4564
```

Problem T9

Plot the RoC curve for this simple verification system.

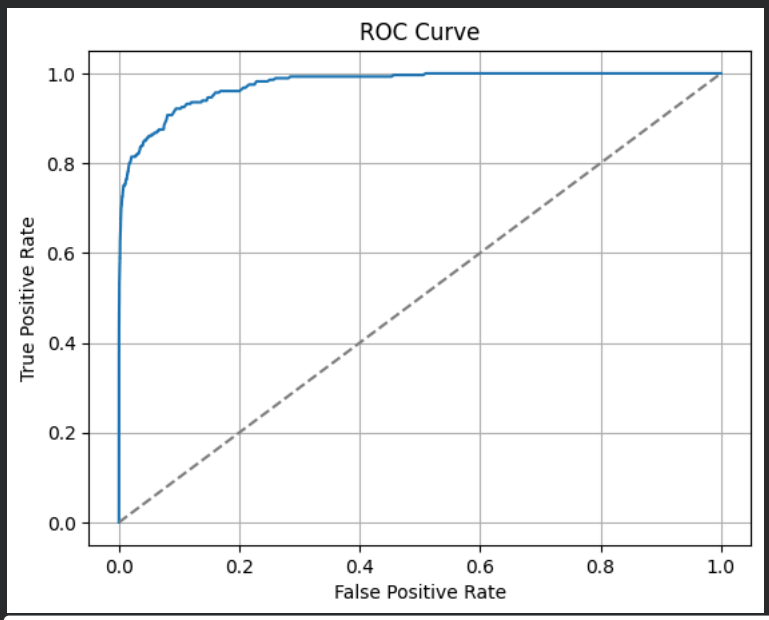
What should be the minimum threshold to generate the RoC curve?

What should be the maximum threshold? Your RoC should be generated from at least 1000 threshold levels equally spaced between the minimum and the maximum. (You should write a function for this).

```
def calculate_roc(input_mat):
    """
    Calculate a list of true_pos_rate and a list of false_pos_rate from the given matrix.
    """
    min_threshold = np.min(input_mat)
    max_threshold = np.max(input_mat) + 0.1

    thresholds = np.linspace(min_threshold, max_threshold, 1000)

    tpr_list = []
    fpr_list = []

    for t in thresholds:
        tpr, fpr = evaluate_performance(input_mat, t)
        tpr_list.append(tpr)
        fpr_list.append(fpr)

    return tpr_list, fpr_list
```

```
def plot_roc(input_mat):
    """
    Plot RoC Curve from a given matrix.
    """
    tpr_list, fpr_list = calculate_roc(input_mat)

    plt.figure()
    plt.plot(fpr_list, tpr_list)
    plt.plot([0, 1], [0, 1], color="gray", linestyle="--")

    plt.title("ROC Curve")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")

    plt.grid(True)

    plt.show()
```

```
plot_roc(similarity_matrix)
```



Answer

- **Minimum Threshold** should be 0 or the minimum value in the similarity threshold.
- **Maximum Threshold** should be maximum value in the similarity threshold.

Problem `T10`

What is the EER (Equal Error Rate)?

What is the recall rate at 0.1% false alarm rate? (Write this in the same function as the previous question)

```
def calculate_metrics(tpr_list, fpr_list, thresholds):
    tpr = np.array(tpr_list)
    fpr = np.array(fpr_list)
    thresh = np.array(thresholds)

    eer_idx = np.argmin(np.abs(fpr - (1 - tpr)))

    eer = (fpr[eer_idx] + (1 - tpr[eer_idx])) / 2
    eer_thresh = thresh[eer_idx]

    TARGET_FPR = 0.001

    fpr_idx = np.argmin(np.abs(fpr - TARGET_FPR))
    recall = tpr[fpr_idx]

    return eer, 1 - eer, recall, eer_thresh
```

```
tpr, fpr = calculate_roc(similarity_matrix)
THRESHOLDS = np.linspace(0, 15, 1000)

eer, verification_rate, recall, eer_thresh = (
    calculate_metrics(tpr, fpr, THRESHOLDS)
)

print(f"Equal Error Rate (EER): {eer:.4f}")
print(f"Verification Rate: {verification_rate:.4f}")
print(f"Recall at 0.1% FPR: {recall:.4f}")
```

```
Equal Error Rate (EER): 0.0896
Verification Rate: 0.9104
Recall at 0.1% FPR: 0.5464
```

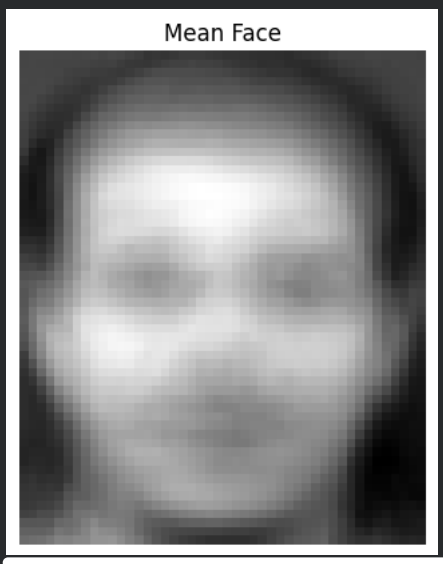
Problem `T11`

Compute the mean vector from the training images. Show the vector as an image (use numpy.reshape()).

This is typically called the meanface (or meanvoice for speech signals). You answer should look exactly like the image shown below.

```
T_raw = xf[:, 0:3]
n_people, n_images, height, width = T_raw.shape

X_train = T_raw.reshape(n_people * n_images, height, width)

mean_vector = np.mean(X_train, axis=0)
mean_face = mean_vector.reshape(height, width)
```

```
plt.imshow(mean_face, cmap="gray")
plt.title("Mean Face")
plt.axis("off")
plt.show()
```



Problem T12

What is the size of the covariance matrix? What is the rank of the covariance matrix?

```
n_features = height * width
n_samples = 40 * 3

cov_matrix_size = (n_features, n_features)
cov_matrix_rank = min(n_samples - 1, n_features)

print(f"Covariance Matrix Size {cov_matrix_size}")
print(f"Covariance Matrix Rank {cov_matrix_rank}")
```

```
Covariance Matrix Size (2576, 2576)
Covariance Matrix Rank 119
```

Problem T13

What is the size of the Gram matrix?

What is the rank of Gram matrix? If we compute the eigenvalues from the Gram matrix, how many non- zero eigenvalues do we expect to get?

```
X_train_flat = T_raw.reshape(120, -1)
X = X_train_flat.T

mean_vec_col = mean_vector.reshape(-1, 1)
X_hat = X - mean_vec_col

gram_matrix = np.dot(X_hat.T, X_hat)
```

```
gram_size = gram_matrix.shape
gram_rank = np.linalg.matrix_rank(gram_matrix)

print(f"Gram Matrix Size {gram_size}")
print(f"Gram Matrix Rank {gram_rank}")
```

```
Gram Matrix Size (120, 120)
Gram Matrix Rank 119
```

Problem `T14`

Is the Gram matrix also symmetric? Why?

```
is_symmetric = np.allclose(gram_matrix, gram_matrix.T)
print(f"The Gram matrix is symmetric: {is_symmetric}")
```

```
The Gram matrix is symmetric: True
```

Answer

The Gram matrix is symmetric, because each member is the dot product between two members which has the commutative property